Vivva Admin - Full-Stack Project

Product Specification Document - Vivva Admin

1. Problem the Product Solves
   Small vintage/resale sellers (operating via Instagram, pop-ups and marketplaces) currently track inventory, pricing, and sales manually in spreadsheets or notebooks. This makes it difficult to know what's actually profitable, what's overstocked, and slows down bookkeeping and tax preparation.

2. Who the Users Are
   Owners of small resale/retail businesses who manage physical inventory and need lightweight back-office tools without the overhead of enterprise POS systems. Each user manages their own isolated business and inventory data within the platform.

3. Who the Customer Is
   The customer is the same as the user: small business owners who sign up and use the tool directly to manage their own business (self-serve, multi-tenant model — each business owner is an independent paying/using account).

4. Business Goals of the Product
   - Give owners real visibility into what's selling versus what's sitting unsold (sell-through rate)
   - Make margin and profitability visible per product category, not just estimated
   - Reduce manual admin time spent on receipts and bookkeeping preparation
   - Support multiple independent businesses on a single platform

5. Software Capabilities Needed to Enable the Business Goals
   - User authentication with per-user data isolation (Supabase Auth + Row Level Security)
   - Inventory management (create, read, update, delete items — cost, price, category, status: in-stock/sold), scoped to the logged-in user's business
   - Statistics engine calculating sell-through rate and margin, aggregated by product category
   - Receipt generation (PDF or printable) tied to a completed sale
   - Accountant-ready report export (CSV/PDF) summarizing sales, costs, and margins over a selected date range

6. Core Processes the Product Enables Users to Preform
   1. Sign up and log in

2. Add an inventory item (cost, price, category)
3. Mark an item as sold, generating a receipt
4. View a dashboard showing sell-through rate and margin by category
5. Export an accountant-ready report for a selected date range

Technical Architecture Plan - Vivva Admin

1. System Components
    - Frontend: Next.js (App Router) + TypeScript, deployed on Vercel
    - Backend: Next.js Server Actions / API Routes (same codebase, no separate backend service)
    - Database & Auth: Supabase (Postgres + Supabase Auth + Row Level Security)
    - Hosting: Vercel (app), Supabase (database)

2. Database Usage
   Yes, Supabase Postgres, accessed via Supabase's JS client from server-side code (Server Actions/API Routes), with Row Level Security enforcing per-user data isolation.

3. Core Tables/Entities
    - Users: managed by Supabase Auth
    - Businesses: one per user (business name, owner_id → user)
    - Categories: belongs to a business (name)
    - Items: belongs to a business and category (name, cost, price, status: in_stock/sold, created_at, sold_at)
    - Receipts: belongs to an item/sale (receipt_number, item_id, sale_price, created_at)

4. Application Pages
    - /login, /signup
    - /dashboard - sell-through rate & margin stats by category
    - /inventory - list/add/edit items
    - /inventory/[id] - item detail, mark as sold → generates receipt
    - /receipts - list of past receipts
    - /reports - select date range, export accountant report

5. Server Actions / API Routes Needed
    - createItem, updateItem, deleteItem, markItemSold
    - createCategory, deleteCategory
    - getDashboardStats (sell-through rate + margin by category)
    - generateReceipt
    - exportReport (data range → CSV/PDF)

6. Data Flow Between Frontend, Backend and Database
   User interacts with a page (e.g. marks an item sold) → the page calls a **Server Action** → the Server Action calls **Supabase** (with the user's session) to read/write the relevant table → Postgres enforces **RLS** so only that user's rows are touched → the Server Action returns updated data → the UI re-renders (React Server Components / revalidation, no separate client-side fetch layer needed for most flows).

7. Users and Permissions
   - Single role: **business owner**. Each authenticated user can only read/write rows belonging to their own business_id (enforced via Supabase RLS policies keyed on auth.uid()).
   - No admin/staff roles in MVP - every business is fully self-contained and isolated from every other business.

8. External Libraries / Services
   - Supabase: database, auth, RLS (chosen because it's the assignment's required stack and removes the need to build custom auth)
   - Vercel: hosting/deployment (assignment requirement, zero-config for Next.js)
   - A PDF Library: e.g @react-pdf/renderer or pdf-lib) - for receipts and report generation
   - Optional: a charting library (e.g Recharts) - for the dashboard's sell-through/margin visualizations

Technical Plan - Vivva Admin

1. Project Folder Structure

```
/app
    /login
    /signup
    /dashboard
    /inventory
        /[id]
    /receipts
    /actions                -> Server Actions (createItem, markItemSold, etc.)
/components
    /ui                     -> shared UI (Button, Input, Modal, Table)
    /inventory              -> ItemCard, ItemForm, ItemList
    /dashboard              -> StatsCard, MarginChart
    /receipts               -> ReceiptView, ReceiptPDF
/lib
    /supabase               -> client setup (server + browser clients)
    /types                  -> shared TypeScript types
/middleware.ts              -> route protection (redirect if not loffed in)
```

2. Core Component Structure
   - layout components: AppShell (nav + auth-aware layout wrapper)
   - inventory: ItemList (table/grid of items) → ItemCard (single item) → ItemForm (add/edit modal)
   - dashboard: StatsSummary (top-level numbers) → CategoryBreakdown (per-category sell-through/margin)
   - receipts: ReceiptView (renders a single receipt) → triggered from ItemCard on "mark sold"
   - reports: DateRangePicker + ExportButton

3. Database Structure

Table	Key Columns
businesses	id, user_id (FK → auth.users), name, created_at
categories	id, business_id (FK), name
items	id, business_id (FK), category_id (FK), name, cost, price, status (in_stock/sold), created_at,sold_at
receipts	id, item_id (FK), business_id (FK), sale_price, receipt_number, created_at

RLS policy pattern applied to every table: business_id IN (SELECT id FROM businesses WHERE user_id = auth.uid()

Note: Receipts are immutable once created — no update or delete operations are permitted on this table (enforced via RLS), mirroring standard accounting practice where receipts shouldn't be alterable after issuance.

4. Core CREATE / READ / UPDATE / DELETE Operations
    - create: new business (on signup), new category, new item
    - read: list items (filtered/sorted), get dashboard stats, list receipts, get report for date range
    - update: edit item, mark item as sold (status + sold_at + triggers receipt creation)
    - delete: delete item, delete category (only if no items reference it)

5. API Description
   All data operations go through **Server Actions** (no separate REST API needed):
    - createItem(data) → inserts row, returns new item
    - updateItem(data) → updates row
    - markItemSold(id, salePrice) → updates status + creates receipt in one transaction
    - deleteItem(id) → deletes row
    - getDashboardStats(businessId) → aggregate query (sell-through %, margin by category)
    - exportReport(startDate, endDate) → queries sold items in range, returns CSV/PDF buffer

6. Core Business Logic
    - Sell-through rate (per category) = sold_items / total_items * 100
    - Margin (per item) = price - cost; **margin %** = (price - cost) / price * 100
    - Marking sold must atomically: update item status, set sold_at, and create a receipt row — so a partial failure never leaves an item "sold" with no receipt

7. State Management
    - Mostly **server state** - Server Components fetch fresh data on each navigation; Server Actions trigger revalidatePath to refresh data after mutations
    - Minimal **client state** (userState) only for local UI concerns: form inputs, modal open/close, date range picker selection
    - No global state library needed (no Redux/Zustand) - the app's data needs don't justify the added complexity

8. Error Handling
    - Server Actions return { success, error } shapes rather than throwing raw exceptions to the UI
    - Form-level errors shown inline near relevant field

- Failed mutations (e.g. network/database error) show a toast/banner, and don't optimistically update the UI until confirmed
- RLS violations (shouldn't happen via UI, but as a safety net) are caught and logged, shown as a generic "permission denied" message

9. Input Validation
    - Client-side: required fields, positive numbers for cost/price, price >= 0
    - Server-side (never trust the client): re-validate all fields in Server Action before touching the database — using a schema library (e.g. Zod) to define and enforce shapes for item/category/receipt inputs

10. Core UX Planning
    - Inventory list: the default landing view after dashboard — fast add-item flow (minimal required fields: name, cost, price, category)
    - Marking sold: a single click/confirm, not a full form — should feel fast since it's the highest-frequency action
    - Dashboard: shows the "why does this matter" numbers first (sell-through, margin) before drilling into raw inventory
    - Empty states matter: a new user's first screen (no items yet) should clearly prompt "add your first item," not just show an empty table

Test Specification Document - Vivva Admin

1. Core Feature Tests
    - User can sign up and log in successfully
    - User can create, edit and delete an inventory item
    - User can create and delete a category
    - User can mark an item as sold and a receipt is generated
    - Dashboard correctly displays sell-through rate and margin by category
    - User can export an accountant report for a selected date range

2. Invalid Input Tests
    - Creating an item with negative or zero price is rejected
    - creating an item with cost > price is allowed (loss item) but flagged/handled, not silently broken
    - Creating an item with missing required fields (name, category) is rejected
    - Non-numeric input in cost/price fields is rejected
    - Selecting an invalid/reversed date range for a report (end date before start date) is rejected

3. Core Business Process Tests
    - Marking an item “sold” updates its status **and** creates exactly one receipt (never zero, never duplicate)
    - Sell-through rate recalculates correctly after an item’s status changes
    - Margin calculation is correct for a range of price/cost combinations, including zero-margin edge case
    - Export report totals match the sum of individual sold items in that date range

4. Permission / Authorization Tests
    - A logged-out user cannot access /dashboard, /inventory, /receipts, or /reports (redirected to login)
    - User A cannot view, edit or delete User B’s items, categories or receipts - even via direct API/URL manipulation (e.g guessing an item ID)
    - A user can only see their own business’s data on the dashboard and reports

5. Database Tests
    - RLS policies correctly block cross-tenant reads/writes at the database level (tested independently of the UI, e.g. via direct query with another user’s session)
    - Deleting a category that still has items attached is prevented or handled per the defined rule (e.g. block deletion or reassign items)
    - Foreign key constraints prevent orphaned records (e.g. a receipt without a valid item)

6. Edge Case Tests
    - Marking the same item as sold twice in quick succession does not create duplicate receipts
    - Dashboard stats behave correctly when a business has zero items (no divide-by-zero crash on sell through %)
    - Report export for a date range with zero sales returns an empty/valid report, not an error
    - Very large cost/price values don't break calculations or UI display

7. Basic UI Tests
    - Add Item form shows validation errors inline for invalid input
    - Empty inventory state shows a clear prompt to add the first item
    - Navigation between dashboard, inventory, receipts, and reports works without errors
    - Mark-as-sold action gives clear visual confirmation (e.g. status change, toast, or redirect to receipt)

Basic Scale Document - Vivva Admin

What happens with dozens/hundreds of users: Since every business's data is isolated by business_id and each business likely holds a few hundred items at most, the app should perform fine at this scale on Supabase's default Postgres tier — the real risk isn't total user count, it's an individual business's item count growing unbounded on an unpaginated list.

Heaviest potential database queries: The dashboard's sell-through/margin aggregation (scanning all items per business) and the report export (scanning all sold items in a date range) are the two queries that do real work — both should be scoped tightly by business_id and indexed accordingly.

Indexes needed: Index business_id on items, categories, and receipts (since every query filters by it via RLS), plus a composite index on items(business_id, status) for fast sell-through calculations, and items(business_id, sold_at) for date-range report queries.

Avoiding unnecessary data loading: Inventory list and reports should only ever fetch the current business's rows (enforced by RLS as a safety net, but also explicitly filtered in queries for performance) — never fetch-all-then-filter-client-side.

Pagination: The inventory list and receipts list should use pagination (e.g. 25–50 items per page) once a business has more than a page's worth of items, rather than loading the full inventory on every visit.

Client/server separation: Data fetching and mutations happen server-side (Server Components + Server Actions); the client only holds transient UI state — this avoids shipping large datasets to the browser unnecessarily and keeps sensitive queries off the client entirely.

Current limitations: No caching layer beyond Next.js's built-in revalidation; dashboard stats are computed live on each load rather than pre-aggregated; no background jobs for report generation (synchronous, fine at small scale).

Future improvements for larger scale: Add a materialized view or scheduled job to pre-compute dashboard stats instead of live aggregation; move large report exports to an async/background job with a "download when ready" flow; add caching (e.g. Redis or Next.js data cache) for frequently-viewed dashboard data.

Basic Security Document - Vivva Admin

Authentication: Handled entirely by Supabase Auth (email/password) — no custom auth logic is written; sessions are managed via Supabase's session tokens.

Authorization: Enforced primarily at the database level via Row Level Security — every table's policies check that business_id belongs to the currently authenticated user (auth.uid()), not just in application code.

Logged-in-only actions: All inventory, receipt, and report actions require an active session; middleware.ts redirects unauthenticated users to /login before they can reach any protected route.

Preventing access to other users' data: RLS policies are the real enforcement layer — even if a bug in the UI or a manipulated request tries to read/write another business's row, Postgres itself rejects it, since access isn't just gated by application logic.

Input validation: All inputs are validated server-side in Server Actions (using a schema library like Zod) before touching the database — client-side validation exists only for UX (instant feedback), never trusted as the actual security boundary.

Protecting API calls: Since there's no separate public API (only Server Actions, which are not directly callable from outside the app in the same way a REST endpoint is), the main protection is the session check + RLS on every action — no action trusts a business_id passed from the client without verifying it against the authenticated session.

Storing secrets: Supabase API keys and any other secrets are stored in environment variables (.env.local locally, Vercel's environment variable settings in production) — never committed to the repository or exposed in client-side code.

Remaining security risks / future improvements: No rate limiting on auth endpoints (vulnerable to brute-force login attempts at scale); no audit log of who changed what; no email verification enforcement in MVP; would add rate limiting, 2FA as an option, and structured audit logging in a future version.

*certain records can't be modified once created, by design